Aurore et le trésor d'Aurore

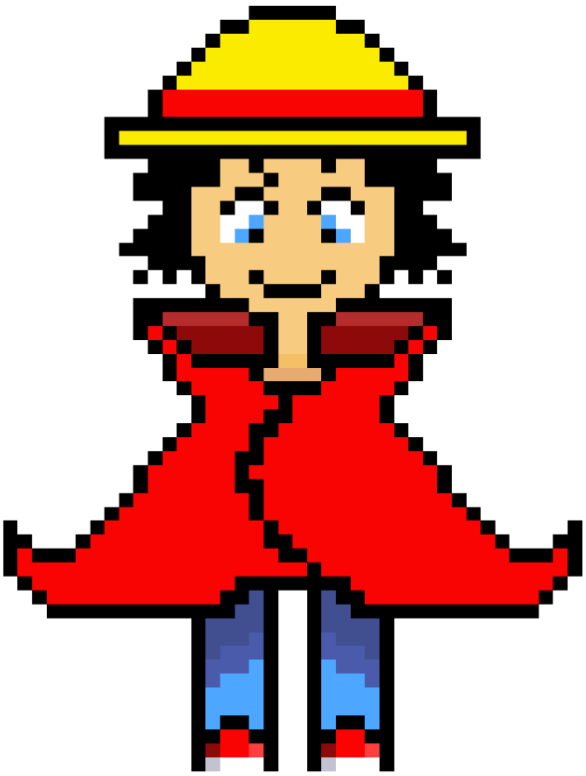

Figure 1 : image de Titre

1 Table des matières

1 Table des matières 2

2 Analyse préliminaire 4
2.1 Introduction 4
2.2 Objectifs 4
2.3 Planification initiale 6

3 Analyse / Conception 6
3.1 Concept 6
3.2 Maquettes : 7
3.3 Diagramme de Classe : 9
3.4 Choix du code : 10
3.5 Stratégie de test 11
3.6 Risques techniques 12
3.7 Planification 13
3.8 Dossier de conception 13

4 Réalisation 14
4.1 Dossier de réalisation 14
4.2 Description des tests effectués plus résultats des tests d'acceptations 15
4.2.1 Mouvement du joueur (flèche du haut) : 15
4.2.2 Mouvement du joueur (flèche du bas) : 16
4.2.3 Mouvement du joueur (flèche de gauche) : 16
4.2.4 Mouvement du joueur (flèche de droite) : 16
4.2.5 Collision entre le joueur et les bords de l'écran : 17
4.2.6 Collision entre le joueur et les objets : 17
4.2.7 Ennemi qui suit le joueur lorsqu'il est proche par le haut : 18
4.2.8 Ennemi qui suit le joueur par le bas : 18
4.2.9 Ennemi qui suit le joueur par la gauche : 19
4.2.10 Ennemi qui suit le joueur lorsqu'il est proche par la droite : 19
4.2.11 Collision entre l'ennemi et le joueur : 20
4.2.12 Ennemi qui se trouve loin du joueur : 21
4.2.13 Changement de map depuis le haut de l'écran : 21
4.2.14 Changement de map depuis le bas de l'écran : 22
4.2.15 Changement de map depuis la gauche de l'écran : 22
4.2.16 Changement de map depuis la droite de l'écran : 23

4.2.17 Map non connecté par le haut : ... 23
4.2.18 Map non connecté par le bas : ... 24
4.2.19 Map non connecté par la gauche : ... 24
4.2.20 Map non connecté par la droite : ... 25
4.2.21 Affichage de l'inventaire : ... 25
4.2.22 Inventaire vide : ... 26
4.2.23 Sélection des items dans l'inventaire : ... 26
4.2.24 Utilisation de l'item puis disparition de celui-ci : ... 27
4.2.25 Ramassage des items plus stockage dans l'inventaire : ... 27
4.2.26 Equiper une arme : ... 28
4.2.27 Déséquiper une arme : ... 28
4.2.28 L'attaque du joueur : ... 29
4.2.29 Utiliser une potion : ... 30
4.3 Codes Particuliers ... 30
4.3.1 Ennemi qui suit le Joueur : ... 30
4.3.2 Changement de map : ... 32
4.3.3 Les Collisions avec hitbox : ... 33
4.4 Erreurs restantes ... 35
4.5 Amélioration Possible ... 35
4.6 Liste des documents fournis ... 35

5 Conclusions ... 36

6 Annexes ... 37
6.1 Résumé du rapport du TPI / version succincte de la documentation ... 38
6.2 Sources – Bibliographie ... 39
6.3 Journal de travail ... 39
6.4 ... 41
6.5 Manuel d'Installation ... 41
6.6 Manuel d'Utilisation ... 41

2 Analyse préliminaire

2.1 Introduction

Le monde du jeu vidéo est un monde très vaste qui regroupe plusieurs genres et plusieurs types de joueurs. Plusieurs personnes passent le temps à jouer pendant le temps sans se soucier du temps et de la quantité de travail que cela a dû être pour les développeurs et les designers de concevoir le jeu, peu importe le genre.

C'est dans cette optique que j'ai décidé de réaliser mon Pré-TPI afin de me mettre à la place des développeurs pour juger la quantité de travail et ainsi avoir une idée de si la pression que plusieurs grandes entreprises du jeu vidéo infligent à leurs développeurs est justifiée ou non.

Ce projet consiste donc à créer un jeu vidéo en 2D dans lequel nous incarnerons une aventurière à la recherche d'un trésor en explorant un monde rempli d'ennemis et d'énigmes. Un jeu qui me permettra de voir à quel point créer un jeu est un projet ambitieux même si celui-ci est petit.

2.2 Objectifs

Les objectifs de ce projet seront :

- Avoir un système de déplacement avec les touches fléchées
  - Avoir une animation du joueur lors du déplacement

- Avoir un système de points de vie
  - Afficher la barre de points de vie
  - Pouvoir prendre des dégâts diminuant ainsi les points de vie
  - Mourir lorsque les points de vie tombent à zéro
  - Pouvoir se soigner

- Avoir un système d'inventaire
  - Pouvoir ramasser des items et les garder avec soi
  - Pouvoir sélectionner un item spécifique
  - Pouvoir utiliser les items stockés dans l'inventaire
  - Pouvoir équiper un item

- Avoir un système de collision
  - Ne pas pouvoir traverser les murs, les objets et les bords de l'écran
  - Ne pas pouvoir traverser les ennemis avec le système de « hitbox »
  - Gérer les interactions avec les portes qui s'ouvrent

- Avoir un système d'attaque
  - Pouvoir attaquer au contact

    - Dégâts variables selon l'arme

- Avoir un système de changement de map

- Avoir un système de sauvegarde
    - Sauvegarder manuellement via le menu
    - Sauvegarder automatiquement à chaque changement de map
    - Charger la dernière sauvegarde au démarrage

- Avoir un menu principal contenant
    - Le titre du jeu
    - Un bouton New Game
    - Un bouton Continue
    - Un bouton Quitter

- Avoir un menu pause contenant
    - Un bouton Reprendre
    - Un bouton Contrôles
    - Un bouton Sauvegarder
    - Un bouton Quitter

2.3 Planification initiale

Pour ma planification initiale j'ai prévu de commencer par les designs du joueur qui sera animé et du premier background afin de pouvoir directement commencer à coder la base c'est-à-dire le joueur en lui-même ensuite dès que celui-ci sera fait je m'attaquerai à la logique des interactions avec les différents objets ainsi que du menu, des collisions et du changement de map et pour finir je terminerai par la sauvegarde.

Tâches	Sprint 1 (02.02.2026 - 15.02.2026)	Sprint 2 (16.02.2026 - 01.03.2026)	Sprint 3 (11.03.2026 -25.03.2026)
Designe des enemis du joueur et du monde ainsi que des objects	X		
Déplacement du joueur	X		
Attaque du joueur	X		
Inventaire du joueur	X		
interaction du joueur avec les objets	X		
Gestion des point de vie	X		
Ajout des collisions		X	
Ajouts de la logique des interactions des objets		X	
Changement de map		X	
Sauvegarde du monde			X

Figure 2 : Planification Initial

3 Analyse / Conception

3.1 Concept

Pour ce projet, plusieurs fonctionnalités ont été discutées. Parmi celles discutées, voici celles qui ont été choisies :

1. Mouvement du joueur :
   Le joueur pourra se déplacer sur l'écran en utilisant les touches fléchées du clavier, il ne pourra pas traverser les bords de l'écran

2. Point de vie du joueur
   Le joueur possèdera une barre de vie qui s'affichera en haut à gauche de l'écran. Elle sera limitée à 100 hp comme dans la plupart des jeux

3. Système d'interaction avec l'environnement pour le joueur
   Le jeu aura un système d'interaction avec l'environnement pour que le joueur puisse acquérir différents objets ou débloquer de nouvelles portes

4. Système d'attaque pour le joueur
   Le jeu comportera un système d'attaque pour que le joueur puisse se défendre lorsque des ennemis s'approchent de lui pour l'attaquer

5. Système de soin
   Le jeu comportera un système de soin pour que le joueur puisse se régénérer s'il a pris des dégâts

6. Système de collision
   Le jeu possèdera un système de collision qui empêchera le joueur de traverser les ennemis et les objets, limitant ainsi ses déplacements.

7. Changement de map pour se déplacer dans le monde
   Le monde sera illustré sous forme de tableau et le joueur pourra s'y déplacer en touchant les bords de l'écran

8. Un Système de sauvegarde
   Le jeu possèdera un système de sauvegarde qui sauvegardera l'état du monde, la position du joueur ses points de vie son inventaire et son équipement

3.2 Maquettes :

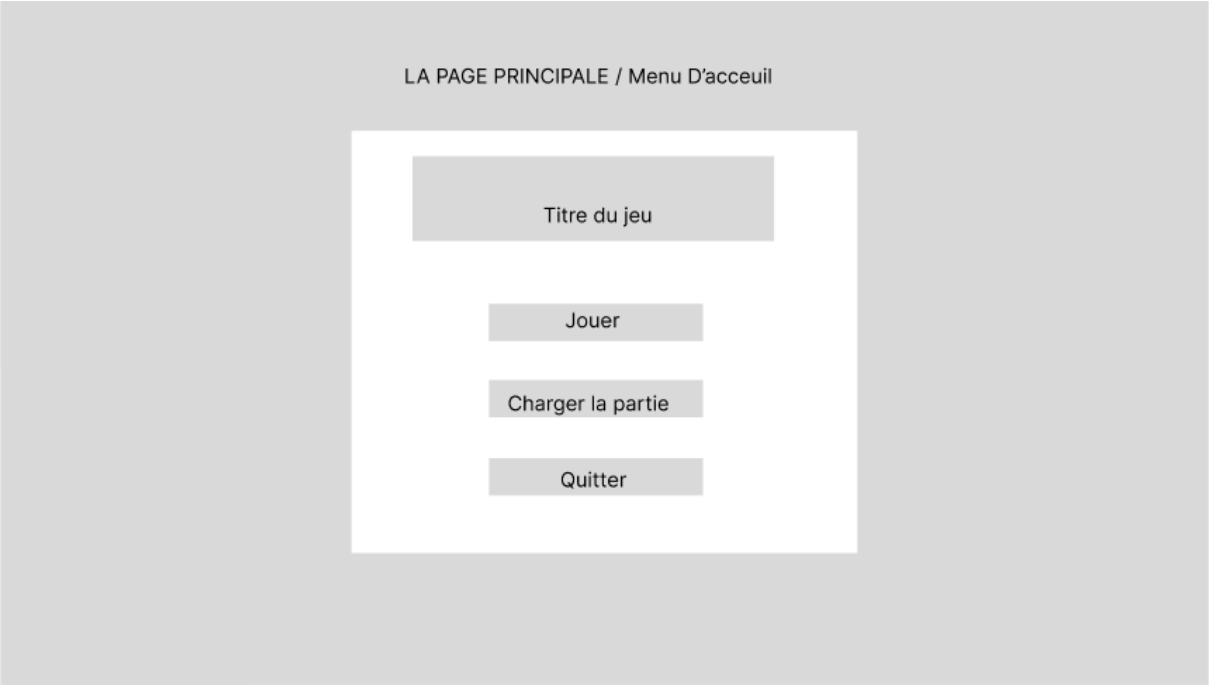

Figure 3 : Maquette du menu d'acceuil

Voici une maquette qui montre à quoi ressemblera le menu principal lorsqu'on lance le jeu.

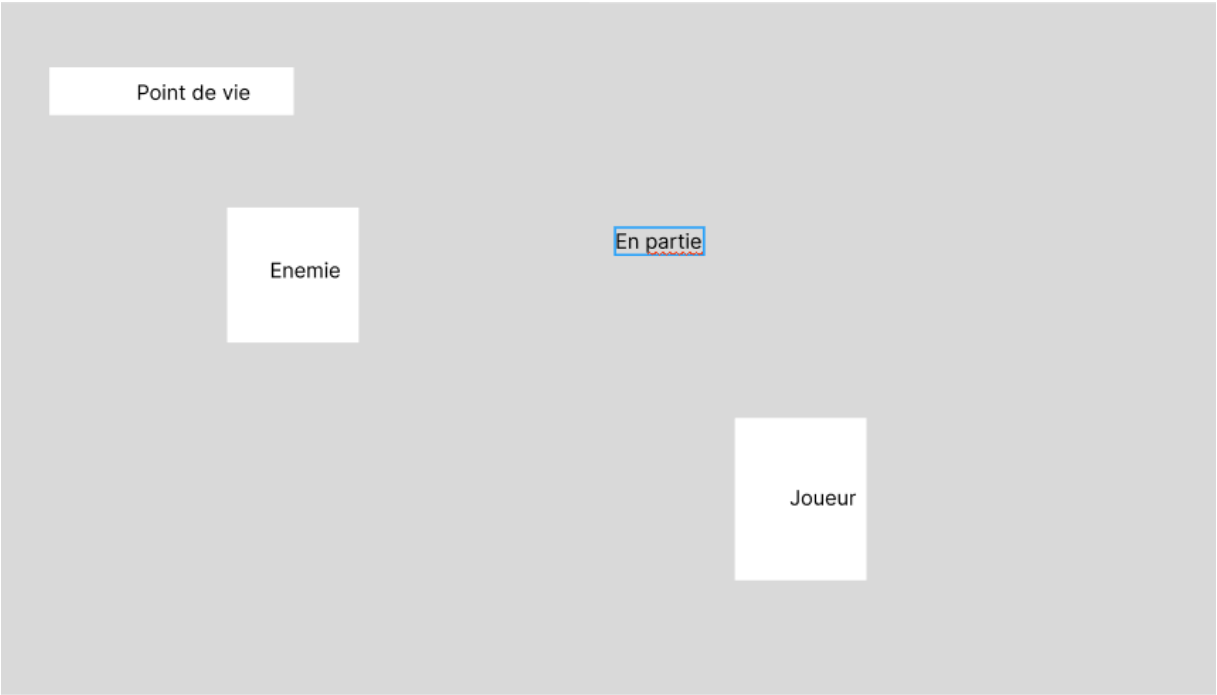

Figure 4 : Maquette du jeu après le menu

Voici la maquette du jeu en pleine partie.

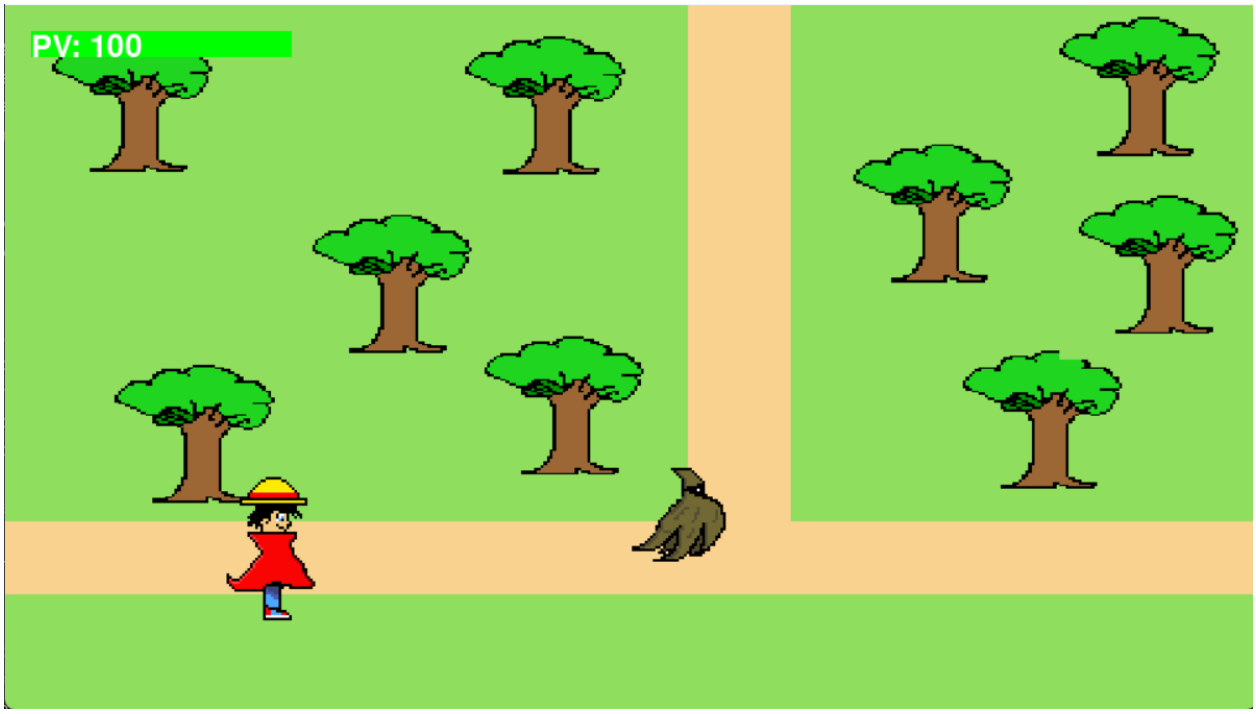

Figure 5 : Représentation plus visuel du jeu

Voici une représentation plus visuelle du jeu en partie.

3.3 Diagramme de Classe :

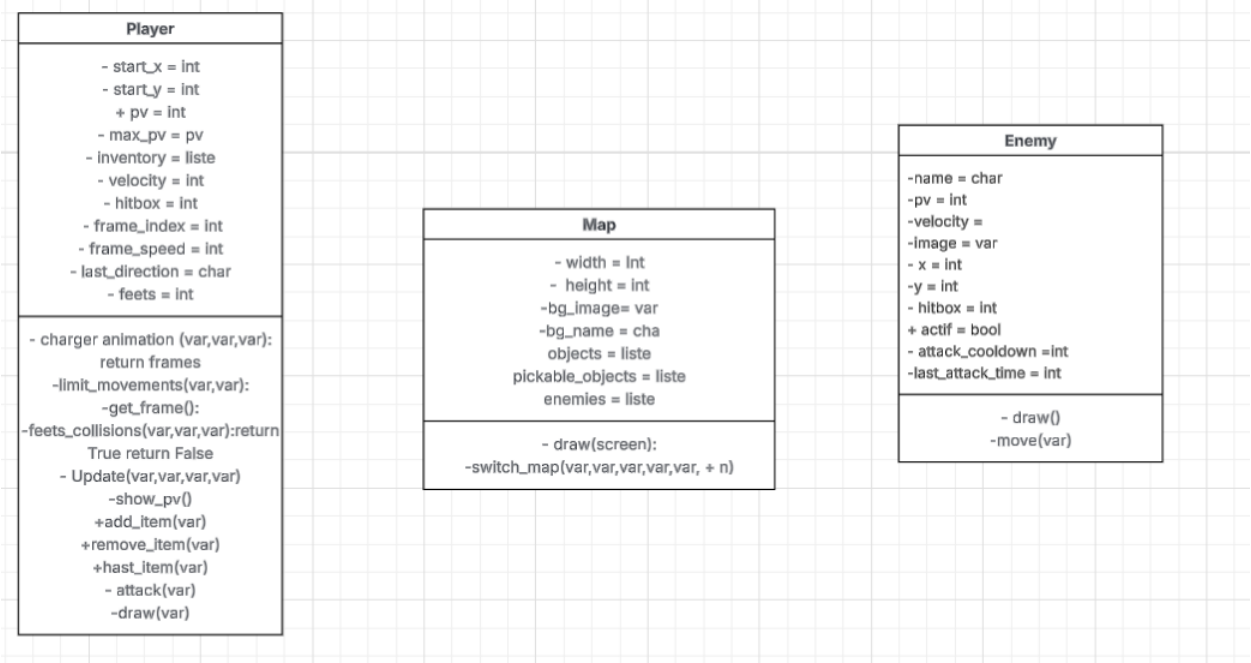

Figure 6 : Diagramme de classes 1

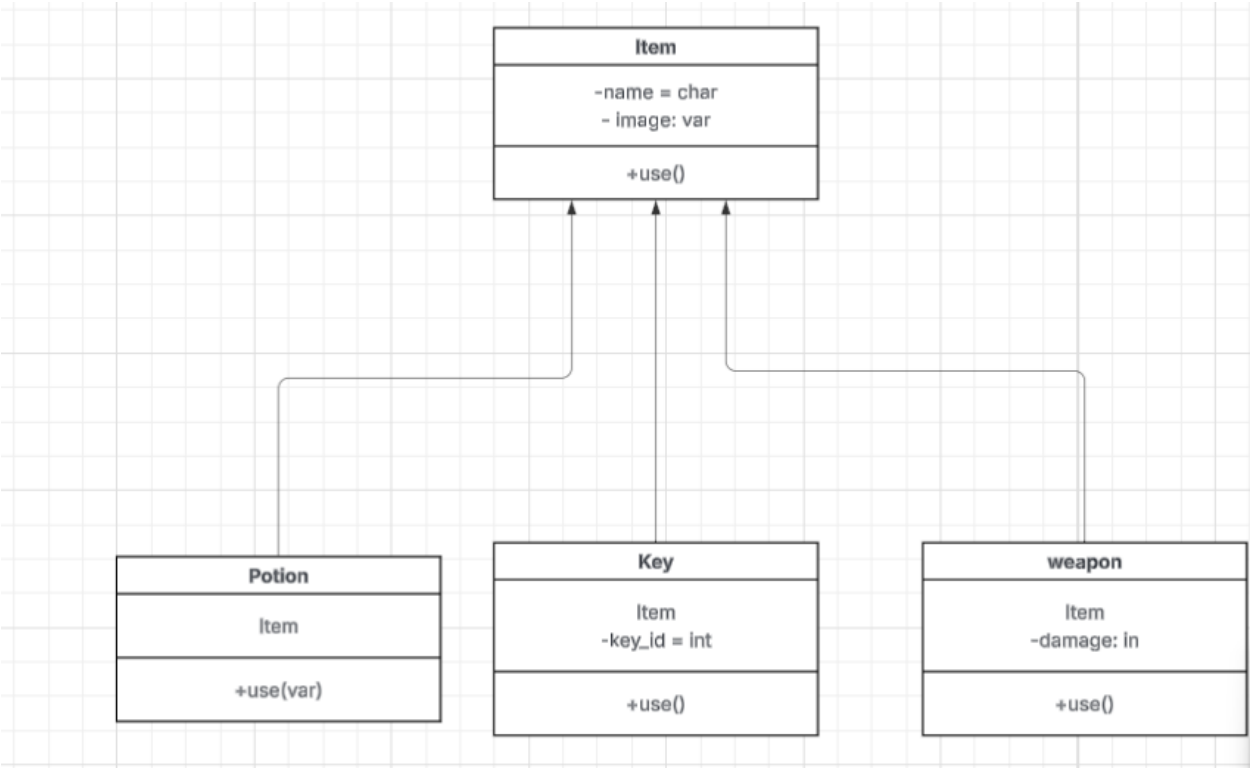

Figure 7 : Diagramme de classes 2

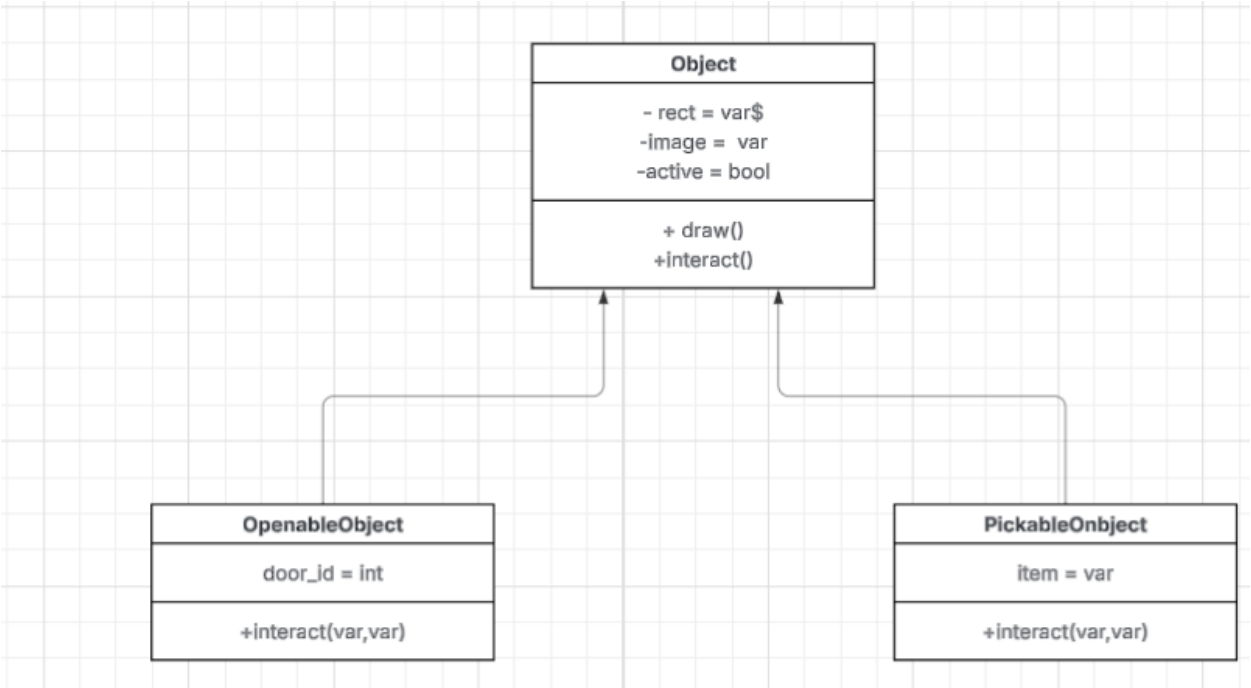

Figure 8 : Diagramme de classes 3

Ces diagrammes ci-dessus illustrent comment les différentes classes du code ont été programmées. On peut voir que la classe Player est la plus conséquente car c'est elle qui, dans un premier temps, interagit avec plusieurs classes comme PickableObject ou Enemy, et dans un deuxième temps car c'est la classe qui représente comme son nom l'indique, Player. C'est elle qui fait le lien entre le jeu et l'utilisateur, sans toutes ses fonctions le jeu ne peut fonctionner.

Ensuite il y a les classes Item et Object qui sont tous deux des classes parents d'autres classes (Key, Potion et Weapon pour Item et OpenableObject PickableObject pour Object). En faisant ça on permet aux classes d'accéder directement aux attributs de leur parent et ainsi d'éviter la redondance d'attributs dans les classes tout en gardant ce qui leur est spécifique. Par exemple pour OpenableObject on lui laisse les attributs de Object qui sont les mêmes pour PickableObject mais on écrit l'attribut « door_id » qui lui est propre.

3.4 Choix du code :

Collision joueur (hitbox) :

J'ai décidé de gérer les collisions du joueur avec le système de hitbox qui permettra de gérer plus facilement les attaques de l'ennemi aussi pour avoir un rendu visuel plus réaliste. Comme la hitbox est un rectangle qui se forme seulement autour de la partie dessinée de le sprite lorsque qu'on la configure et non toute le sprite, on peut l'utiliser pour simuler une collision réaliste avec les ennemis. L'autre avantage de la hitbox est que, comme c'est un rectangle le code sera moins lourd que si on faisait calculer l'ordinateur aux pixels près pour avoir un meilleur résultat.

Déplacement de l'ennemi :

En ce qui concerne les déplacements de l'ennemi, j'utilise la librairie de math pour utiliser la fonction « math.hypot » qui me permettra de calculer la distance du vecteur

entre les deux entités peu importe où elles se trouvent et ainsi pouvoir choisir à partir de quelle distance je veux qu'il se rapproche de la cible.

Changement de map :

Pour le changement de map, le but sera de créer une fonction qui saura reconnaître quelle map est en cours pour qu'elle puisse être liée au bonnes maps. Pour ce faire, j'ai décidé de rentrer le nombre exact de map utilisé en argument pour ensuite créer plusieurs condition if afin de les spécifier pour que le programme puisse savoir quelle map est liée à quelle map afin de sortir la bonne.

3.5 Stratégie de test

En ce qui concerne ma stratégie de test, tous mes tests se feront de manière manuelle à l'exception d'un qui sera automatique qui gérera la version du module Pygame. Pour les autres tests ceux-ci seront manuels car mes fonctions ne peuvent fonctionner que si le jeu n'est pas lancé. Il y aura donc plusieurs tests unitaires pour vérifier le bon fonctionnement de la fonction. Ensuite à la fin un test d'intégration pour tester qu'il n'y a pas de conflits avec les autres fonctions.

Lorsque j'aurai fini d'implémenter les déplacements du joueur je lancerai le jeu et je testerai moi-même les touches pour valider les tests dans les meilleures conditions possible. Puis à la fin du sprint je lance le jeu une dernière fois afin d'être sûr que toutes les fonctionnalités présentes fonctionnent toutes ensemble.

Les tests qui m'aideront à la réalisation de ce projet sont :

- Les déplacements des personnages
  - Déplacement vers le haut
  - Déplacement vers le bas
  - Déplacement vers la gauche
  - Déplacement vers la droite
- Barre de points de vie
  - Affichage de la barre de points
  - Prendre des dégâts
- Collision du joueur avec les bords
  - Collision avec le bord supérieur
  - Collision avec le bord inferieur
  - Collision avec le bord de gauche
  - Collision avec le bord de droite
- Collision du joueur avec les objets du monde
  - Le joueur ne traverse pas les objets du monde
- Déplacement ennemi
  - L'ennemi commence à se déplacer lorsqu'il est proche du joueur
  - Déplacement vers le haut
  - Déplacement vers le bas
  - Déplacement vers la gauche
  - Déplacement vers la droite
  - L'ennemi ne bouge pas quand il est loin du joueur

- Collision avec l'ennemi
  - L'ennemi attaque le joueur
  - L'ennemi ne traverse pas le joueur
- Changement de map
  - Le perso peut changer de map vers le haut si une map est connecté à cet endroit
  - Le perso peut changer de map vers le bas si une map est connecté à cet endroit
  - Le perso peut changer de map vers la gauche si une map est connecté à cet endroit
  - Le perso peut changer de map vers la droite si une map est connecté à cet endroit
  - Le perso ne peut pas changer de map vers le haut si aucune map n'est connecté à cet endroit
  - Le perso ne peut pas changer de map vers le bas si aucune map n'est connecté à cet endroit
  - Le perso ne peut pas changer de map vers la gauche si aucune map n'est connecté à cet endroit
  - Le perso ne peut pas changer de map vers la droite si aucune map n'est connecté à cet endroit
- Récupération des objets
  - Le joueur récupère l'Item et le stocke dans son inventaire
- Système d'inventaire
  - Affichage de l'inventaire
  - Sélectionner un Item de l'inventaire
  - Equiper une arme
  - Déséquiper une arme
  - Utiliser une potion de soin

3.6 Risques techniques

Plusieurs risques peuvent surgir lors de ce projet. Cependant parmi celles-ci, celles qui ont plus de chance de surgir sont :

- Pertes de données si on me vole mon pc ou si tout mon projet est corrompu.
- L'incompatibilité sur macOS ou Linux car je code uniquement sur Windows 10
- Le fait que le client ne puisse pas lancer le jeu, car il se pourrait qu'il n'ait pas installé les dépendances requises

3.7 Planification

Pour le premier sprint je commence par intégrer le design des persos des ennemis et du background afin d'avoir un visuel sur le code et ensuite les fonctionnalités de base du joueur qui sont les déplacements et ses collisions. J'affiche aussi sa barre de vie et je commence à gérer l'ennemi.

Pour le deuxième sprint je commence à agrandir le monde pour intégrer le changement de map, la gestion de l'inventaire, et de l'interaction du joueur avec certains objets ce qui implique la création des objets.

Le troisième sprint regroupera les interactions du joueur avec les items et les ennemis, donc c'est-à-dire, reprendre le système d'inventaire afin qu'il fonctionne pour tous les type d'objets que l'on ajoutera dans le jeu, concevoir les portes pour que le joueur puisse interagir avec et gérer les armes et les attaques du joueur. Ensuite il restera le menu de départ et de pause et enfin la gestion du système de sauvegarde.

3.8 Dossier de conception

OS : Windows 11
Logiciel du design des maps, des ennemis, du joeuur des objets et des items : Pixel Studio version : 5.42.0.0
Logiciel de programmation : JetBrains PyCharm version : 2025.2.1
Langage de programmation : Python version : 3.13.9
Documentation : Word Office version : 2602
Desing de la maquette : Figma version en ligne
Dépôt : Git version 2.51.2
Diagramme de classe : https://lucid.app/lucidchart/decc2c58-4c6f-48be-9ac1-b38fe976cb2d/edit?beaconFlowId=A0E0DDBD1EB3FF73&invitationId=inv_8feadd20-0f56-46a1-935a-df7ab8931388&page=HWEp-vi-RSFO#

Graphique pour illustrer la distance entre l'ennemi et le joueur : https://webwhiteboard.com/
Correction du document : Chat GPT
Définition dans le Glossaire : Gemini

4 Réalisation

4.1 Dossier de réalisation

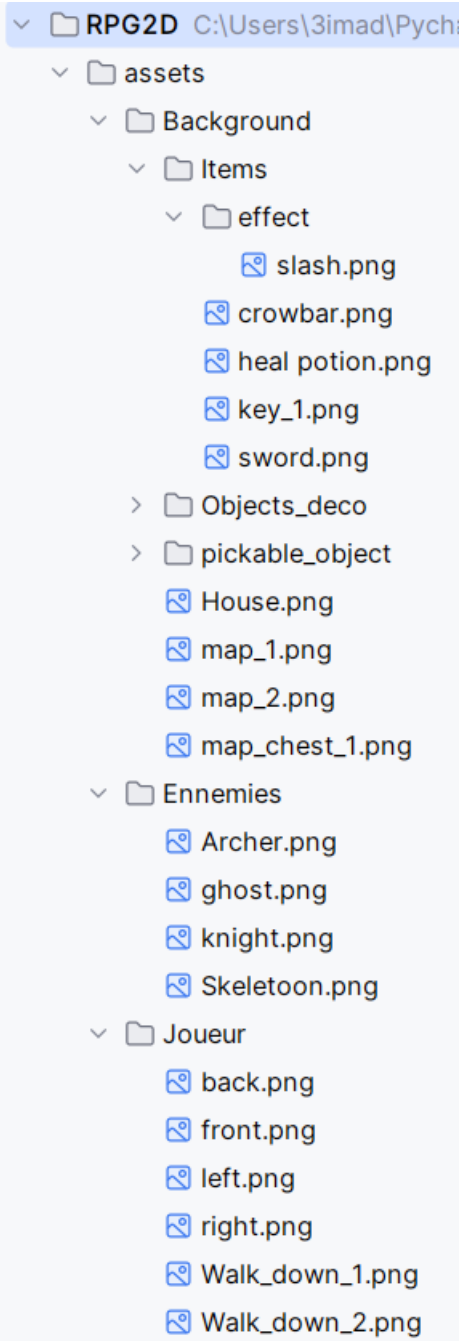

Figure 9 : Arborescence 1

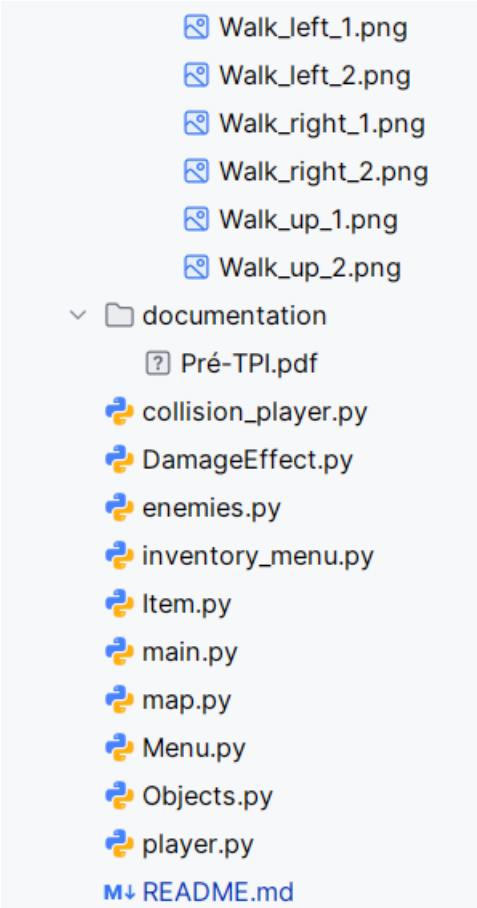

Figure 10 : Arborescence 2

4.2 Description des tests effectués plus résultats des tests d'acceptations

4.2.1 Mouvement du joueur (flèche du haut) :

Description :

Lorsque l'on appuie sur la flèche du haut, le personnage se dirige vers le haut

Test d'acceptation :

Player goes up

Given : The game is started

When : Player press the key arrow up

Then : player goes up

☑ Réussi

Figure 11 : Test du déplacement vers le haut

4.2.2 Mouvement du joueur (flèche du bas) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers le bas

Test d'acceptation :

Player goes down

Given : The game is started

When : Player press the key arrow down

Then : player goes down

☑ Réussi

Figure 12 : Test du déplacement vers le bas

4.2.3 Mouvement du joueur (flèche de gauche) :

Description :

Lorsque l'on appuie sur la flèche gauche, le joueur se dirige vers la gauche.

Test d'acceptation :

Player goes left

Given : The game is started

When : Player press the key left arrow

Then : player goes left

☑ Réussi

Figure 13 : Test du déplacement vers la gauche

4.2.4 Mouvement du joueur (flèche de droite) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers la droite.

Test d'acceptation :

Player goes right

Given : The game is started

When : Player press the key right arrow

Then : player goes right

☑ Réussi

Figure 14 : Test du déplacement vers la droite

4.2.5 Collision entre le joueur et les bords de l'écran :

Description :

Lorsque l'on se dirige vers n'importe quels bords de l'écran non relié vers une autre map, il se retrouve bloqué et ne peut plus continuer dans cette direction.

Test d'acceptation :

Collision with borders

Given :

Player is in the game

When :

He touches the borders of the screen

Then :

He cannot go through

☑ Réussi

Figure 15 : Test des collisions avec les bords de l'écran

4.2.6 Collision entre le joueur et les objets :

Description :

Lorsque le joueur se heurte avec un objet du décor, il ne peut pas le traverser et dois donc changer de direction.

Test d'acceptation :

Collision with objects:

Given :

Player is in the game

When :

He touches any objects in the game

Then :

He cannot go through it

☑ Réussi

Figure 16 : Test des collisions avec les objets

4.2.7 Ennemi qui suit le joueur lorsqu'il est proche par le haut :

Description :

Lorsque l'ennemi se trouve au-dessus du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre

Test d'acceptation :

enemy goes down

Given :

Player and enemy are in the game

When :

player is under and close to the enemy

Then :

the enemy start chasing him by going down

☑ Réussi

Figure 17 : Test du déplacement de l'ennemi vers le bas

4.2.8 Ennemi qui suit le joueur par le bas :

Description :

Lorsque l'ennemi se trouve en dessous du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes up

Given :

Player and enemy are in the game

When :

player is above and close to the enemy

Then :

the enemy start chasing him by going up

☑ Réussi

Figure 18 : Test du déplacement de l'ennemi vers le haut

4.2.9 Ennemi qui suit le joueur par la gauche :

Description :

Lorsque l'ennemi se trouve à gauche du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes left

Given :

Player and enemy are in the game

When :

player is tand close to the enemy

Then :

the enemy start chasing him by going to the left

☑ Réussi

Figure 19 : Test du déplacement de l'ennemi vers la gauche

4.2.10 Ennemi qui suit le joueur lorsqu'il est proche par la droite :

Description :

Lorsque l'ennemi se trouve à droite du joueur et qu'il est suffisamment proche de lui, alors il commence à le poursuivre.

Test d'acceptation :

enemy goes right

Given :

Player and enemy are in the game

When :

player is the right side and close to the enemy

Then :

the enemy start chasing him by going right

☑ Réussi

Figure 20 : Test du déplacement de l'ennemi vers la droite

4.2.11 Collision entre l'ennemi et le joueur :

Description :

Lorsque l'ennemi entre en collision avec le joueur, les deux entités ne peuvent se traverser et l'ennemi attaque le joueur toute les 1 secondes en faisant descendre ses points de vie.

Test d'acceptations :

Collision with ennemies

Given :

Player is in the game

When :

He touches a enemy

Then :

He cannot go through it

☑ Réussi

Figure 21 : Test des collisions avec l'ennemi

enemy touches player

Given :

Player and enemy are in the game

When :

enemy touches player

Then :

player start taking damage

☑ Réussi

Figure 22 : Attaque de l'ennemi en collision avec le joueur

4.2.12 Ennemi qui se trouve loin du joueur :

Description :

Lorsque l'ennemi se trouve loin du joueur, celui-ci ne bouge plus de sa place jusqu'à ce que le joueur se trouve à nouveau proche de lui.

Test d'acceptation :

player is far from enemy

Given :

Player and enemy are in the game

When :

player is far from enemy

Then :

the enemy don't chase the player

☑ Réussi

Figure 23 : Test de l'ennemi qui ne poursuit pas le joueur

4.2.13 Changement de map depuis le haut de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le haut, il peut la traverser et se retrouve sur le bas la nouvelle map.

Test d'acceptation :

Link up to down

Given :

Two background connected by sides

When :

Player go to the top side of one background

Then :

He appears in the next background to the bottom

☑ Réussi

Figure 24 : Test changements de map 1

4.2.14 Changement de map depuis le bas de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le bas, il peut la traverser et se retrouve sur le haut la nouvelle map.

Test d'acceptation :

Link down to up

Given :

Two background connected by sides

When :

Player go to the bottom side of one background

Then :

He appears in the next background to the top

☑ Réussi

Figure 25 : Test changements de map 2

4.2.15 Changement de map depuis la gauche de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par le haut, il peut la traverser et se retrouve sur le côté droite de la nouvelle map.

Test d'acceptation :

Link left to right

Given :

Two background connected by sides

When :

Player go the very left side of one background

Then :

He appears in the next background to the right

☑ Réussi

Figure 26 : Test changements de map 3

4.2.16 Changement de map depuis la droite de l'écran :

Description :

Lorsque le joueur se trouve sur une map connecté par une autre par la droite, il peut la traverser et se retrouve sur le côté gauche de la nouvelle map.

Test d'acceptation :

Link right to left

Given :

Two background connected by sides

When :

Player go the very right side of one background

Then :

He appears in the next background to the left

☑ Réussi

Figure 27 : Test changements de map 4

4.2.17 Map non connecté par le haut :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis le haut, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the top

Given :

A background is not connected to the top

When :

Player go to the top side of the background

Then :

He still next to the top border and the background doesn't change

☑ Réussi

Figure 28 : Test changement de map pas connecté 1

4.2.18 Map non connecté par le bas :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis le bas, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the bottom

Given :

A background is not connected to the bottom

When :

Player go to the bottom side of the background

Then :

He still next to the bottom border and the background doesn't change

☑ Réussi

Figure 29 : Test changement de map pas connecté 2

4.2.19 Map non connecté par la gauche :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis la gauche, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the left

Given :

A background is not connected to the left

When :

Player go to the left side of the background

Then :

He still next to the left border and the background doesn't change

☑ Réussi

Figure 30: Test changement de map pas connecté 3

4.2.20 Map non connecté par la droite :

Description :

Lorsqu'une map n'est pas connecté par une autre depuis la gauche, le joueur est bloqué et ne traverse pas l'écran

Test d'acceptation :

Not linked to the right

Given :

A background is not connected to the right

When :

Player go to the right side of the background

Then :

He still next to the right border and the background doesn't change

☑ Réussi

Figure 31: Test changement de map pas connecté 4

4.2.21 Affichage de l'inventaire :

Description :

Lorsque le joueur appuie sur la touche « I », l'inventaire s'affiche

Test d'acceptation :

Inventory opens

Given :

Player is in the game

When :

Player press the key "I"

Then:

Inventory opens

☑ Réussi

Figure 32 : Test affichage de l'inventaire

4.2.22 Inventaire vide :

Description :

Lorsque l'inventaire est vide et que le joueur l'affiche il y est écrit « Inventaire est vide »

Test d'acceptation :

Empty Inventory

Given :

A player with an empty inventory

When :

Player press the key I

Then :

Inventory opens and says *Inventaire est vide*

☑ Réussi

Figure 33 : Test inventaire vide

4.2.23 Sélection des items dans l'inventaire :

Description :

Le joueur peut sélectionner l'item qu'il souhaite utiliser en les flèches haut et bas du clavier

Test d'acceptation :

Select an Item

Given :

Player is in the Inventory with a lot of Items in it

When :

Player press the keys Up ou Down

Then :

He can choose an Item

☑ Réussi

Figure 34 : Test sélection des Items

4.2.24 Utilisation de l'item puis disparition de celui-ci :

Description :

Lorsque le joueur utilise un item en appuyant sur la touche « E », ce dernier fait effet avant de disparaître de l'inventaire

Test d'acceptation :

Using an Item

Given :

Player is in the Inventory with an Item

When :

Player press the key E

Then :

Player uses the Item and it dissappears

☑ Réussi

Figure 35 : Test Utilisation des Items

4.2.25 Ramassage des items plus stockage dans l'inventaire :

Description :

Lorsque le joueur découvre un Objets ramassable dans le monde il peut le récupérer en appuyant sur la Touche « E ». L'objet disparait du monde et apparait dans l'inventaire.

Test d'acceptation :

Take an Item

Given :

Player is in a background with an Item in the ground

When:

He press "E" to take the Item

Then:

The Item disappears and go in the inventory

☑ Réussi

Figure 36 : Test récupération des Items dans le monde

4.2.26 Equiper une arme :

Description :

Lorsque le joueur a dans son inventaire une arme et qu'il se trouve lui-même dans l'inventaire, il peut en la choisissant et en appuyant sur la touche « E », équiper son arme pour attaquer ensuite.

Test d'acceptation :

Equip a weapon

Given :

Player is in the inventory

When :

Player selects a weapon and press "E"

Then :

Player equipped the weapon selected

☑ Réussi

Figure 37 : Test équiper une arme

4.2.27 Déséquiper une arme :

Description :

Lorsque le joueur a équipé son arme et qu'il se trouve dans l'inventaire, il peut en le sélectionnant et en appuyant sur la touche « E », déséquiper son arme.

Test d'acceptation :

Unequip a weapon

Given :

Player is in the inventory and has equiped a weapon

When :

Player selects the same weapon and press "E"

Then :

Player unequipped the weapon selected

☑ Réussi

Figure 38 : Test déséquiper une arme

4.2.28 L'attaque du joueur :

Description :

Lorsque le joueur a équipé son arme et qu'il appuie sur la touche « X », il peut attaquer un ennemi s'il en rencontre un.

Test d'acceptation :

Player Attack

Given :

Player has equipped a weapon

When :

Player press "X"

Then :

Player attacks

☑ Réussi

Figure 39 : Test attaque du joueur

4.2.29 Utiliser une potion :

Description :

Lorsque le joueur n'a plus l'entièreté de sa vie, il peut aller dans l'inventaire, sélectionner la potion de soin et en appuyant sur la touche « E », reprendre des points de vie.

Test d'acceptation :

Use Healing Potion

Given :

Player is low hp

When :

Goes to the inventory, selects the healiing potion and press "E"

Then:

The potion disappears and player is healed

☑ Réussi

Figure 40 : Test utilisation de la potion de soin

4.3 Codes Particuliers

4.3.1 Ennemi qui suit le Joueur :

Pour faire en sorte que l'ennemi suive le joueur peu importe sa position par rapport au joueur, je me suis aidé de l'intelligence artificielle, qui m'a conseiller d'utiliser la librairie de math déjà intégrée dans Python, pour utiliser la fonction hypoténuse, qui me permettra de calculer la distance entre le joueur et l'ennemi en temps réel et ainsi simuler une poursuite.

```python
def move(self, player):  1 usage (1 dynamic)  riffuu1 *
    if not self.actif:
        return  # ← Ne fait rien si l'ennemi est mort

    dx = player.rect.centerx - self.rect.centerx
    dy = player.rect.centery - self.rect.centery

    distance = math.hypot( *coordinates: dx, dy)
    if distance == 0:
        return

    if distance <= 200:  # évite division par zéro
        dx /= distance
        dy /= distance

        future_hitbox = self.hitbox.copy()
        future_hitbox.x += dx * self.vitesse
        future_hitbox.y += dy * self.vitesse

        if not future_hitbox.colliderect(player.hitbox):
            self.rect.x += dx * self.vitesse
            self.rect.y += dy * self.vitesse

            self.hitbox.x = self.rect.x + 40
            self.hitbox.y = self.rect.y + 70
```

Figure 41: Code du déplacement de l'ennemi

Sur cette image vous pouvez voir une partie du code du déplacement de l'ennemi, l'autre partie étant lorsqu'il attaque le joueur. Le code commence avec le calcul de la distance entre les coordonnées de l'ennemi et celles du joueur. Nous pouvons voir que la fonction math.hypot requiert deux variables qui sont les coordonnées dx et dy qui sont comme indiqués au-dessus, les positions du joueur. Avec ces deux donnés nous pouvons très facilement expliquer ce que fait la fonction par un graphique.

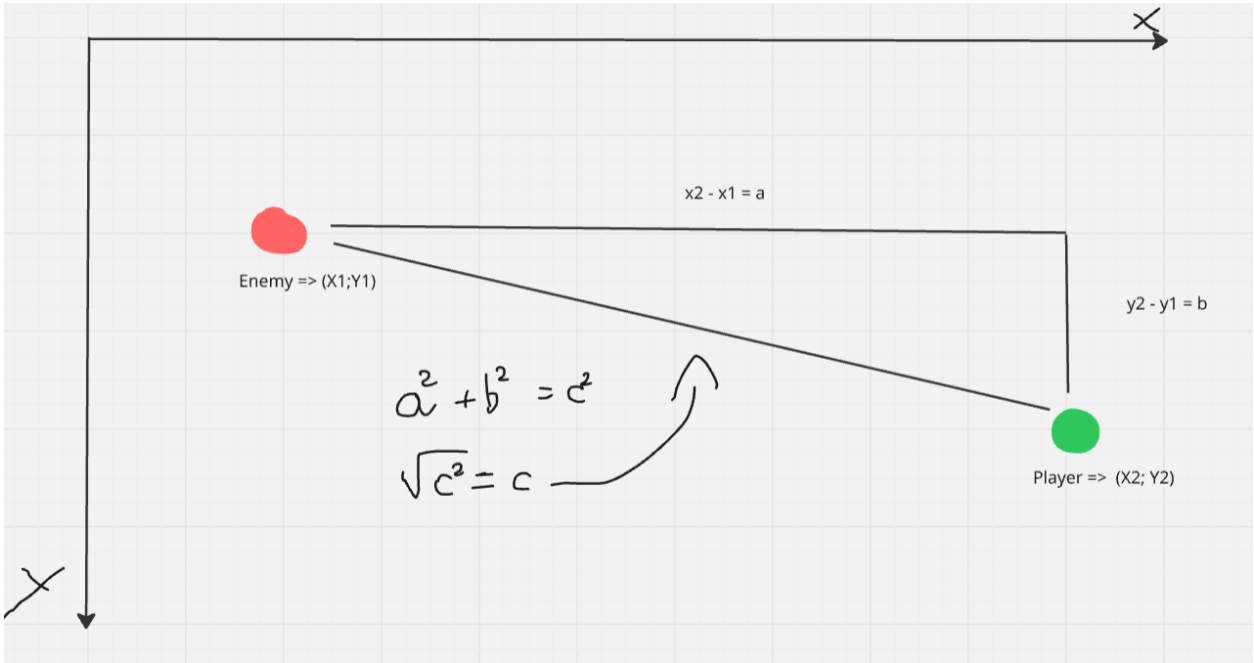

Figure 42: schéma du calcul de la distance

Comme montré sur le graphique la fonction regarde la différence qu'il y a entre la position du joueur, ici marqué en vert, par rapport à l'ennemi, ici marqué en rouge. Comme la fenêtre est un graphique en x et y, avec son ordonnée à l'origine situé en haut à gauche de la fenêtre, il peut ainsi récupérer les coordonnées des deux entités et ainsi réaliser la formule de Pythagore pour trouver comme son nom l'indique, l'hypoténuse et ainsi calculer en tout temps la distance qui le sépare du joueur pour ensuite le poursuivre ou non.

4.3.2 Changement de map :

Lorsque j'ai commencé à tester ma fonction « switch_map » pour changer de map, celle-ci a posé quelques difficultés du fait qu'il fallait être attentif sur la position du joueur ainsi que du background, afin de pouvoir placer la prochaine map voulu. Ces difficultés ont cependant, été résolu assez rapidement pour avoir une personne qui se déplace dans son propre monde.

```
@staticmethod  1 usage  riffuu1 *
def switch_map(current_map,player,map1,map2,map3,House):
    # Sortie à gauche
    if player.rect.x ==0:
        if current_map == map1:
            player.rect.x = map2.width - player.rect.width
            return map2


    # Sortie à droite
    elif player.rect.x == 800:
        if current_map == map2:
            player.rect.x = 0
            return map1

    #Sortie en haut
    elif player.rect.y ==0:
        if current_map == map2:
            player.rect.y = 410
            return map3

    #Sortie en bas
    elif player.rect.y  == 412:
        if current_map == House:
            player.rect.y = 300
            return map1
        if current_map == map3:
            player.rect.y = 0
            return map2
    return current_map
```

Figure 43: Code du changement de map

Cette image illustre la fonction « switch_map ». À l'intérieur de cette fonction on retrouve 4 blocs et chaque bloc représente un sens de direction (gauche, droite, haut, bas). À l'intérieur de ces blocs se trouve ensuite une condition if qui cherche à savoir le background actuel afin de pouvoir utiliser par la suite la prochaine bonne image.

4.3.3 Les Collisions avec hitbox :

Gérer les collisions a été un vrai défi car, au début, j'ai commencé par le faire pixel par pixel ce qui marchait assez bien pour les collisions entre le joueur et les objets mais qui a fini par poser un problème lorsque le joueur entrait en collision avec un ennemi, car à la fin il restait toujours un petit espace entre les deux entités. Pour pallier ce problème, j'ai décidé de changer de méthode de collision et ai opté pour une solution avec un système de hitbox. Le système de hitbox est utilisé dans le monde du jeu vidéo car il permet d'enfermer le personnage du joueur dans une sorte de rectangle qui gèrera les collisions dans son environnement. Dans notre cas il est très utile car lorsque le code prend l'image, il délimite un périmètre dans lequel, il inclue non seulement ce qui a été dessiné mais aussi ce qui n'a pas été dessiné. En d'autres termes, il prend l'image dans son entièreté. En utilisant le système de hitbox, on peut modifier ce périmètre pour qu'il n'inclue que ce que l'on désire, dans notre cas simplement l'image.

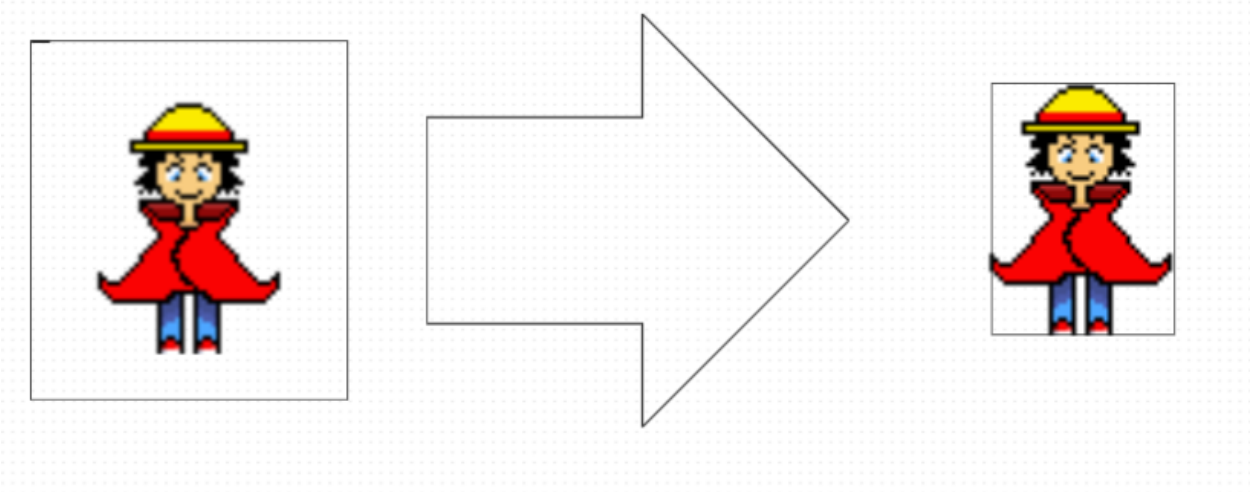

Figure 44: Représentation de la hitbox

L'image que vous voyez au-dessus illustre bien le système de hitbox. On peut voir qu'au début nous avons l'image ainsi que le reste du tableau à gauche et qu'ensuite après la mise en place du système de hitbox le rectangle rétrécit pour avoir les bonnes dimensions de l'image.

Bien sûr il reste encore un peu de vide après la réalisation de la hitbox, cependant ce vide n'est pas important car il ne se trouve pas à un endroit qui pourrait fausser la collision. Alors pourquoi ne pas continuer de le faire pixel par pixel dans ce cas ? D'une part car le temps de travail deviendrait trop long et aussi car le code deviendrait extrêmement lourd si l'ordinateur devait systématiquement calculer une collision au pixel près.

```
self.hitbox = pygame.Rect(self.rect.x + 40, self.rect.y + 70, 48, 50)
```

Figure 45: Initialisation de la hitbox

Ce code est la traduction de l'illustration au-dessus c'est ici que l'on définit clairement la hauteur ainsi que la largeur de la hitbox. Les nombres qui sont rajouté en plus du « self.rect.x » ou « self.rect.y » déterminent le point où va commencer le rectangle et les deux derniers nombres à la fin signife la largeur (48) et la hauteur (50) de la hitbox. Ensuite pour pouvoir utiliser cet attribut, on le place dans la fonction update pour qu'il le calcule constamment lorsque le joueur se déplace.

```
if keys[pygame.K_UP] or keys[pygame.K_w]:
    self.rect.y -= self.vitesse
    self.current_animation = self.animations["walk_up"]
    self.derniere_direction = "back"
    en_mouvement = True
if keys[pygame.K_DOWN] or keys[pygame.K_s]:
    self.rect.y += self.vitesse
    self.current_animation = self.animations["walk_down"]
    self.derniere_direction = "front"
    en_mouvement = True

self.hitbox.y = self.rect.y + 70
self.feet.y = self.rect.y + 118
```

Figure 46: Code exemple d'utilisation hitbox

Dans ce bout de code update on voit tout en bas l'utilisation de la hitbox. À cet endroit on décale la hitbox de sorte qu'il soit constamment à jour lorsque le joueur se déplace. Dans ce cas pour le rectangle on le déplace de 70.

4.4 Erreurs restantes

Bien que dans l'ensemble le projet ait bien avancé, il y a plusieurs fonctionnalités qui n'ont pas pu être programmées à temps. Parmi celles-ci il y a :

- La finalisation des interactions car il n'y a pas encore la clé et les portes qui s'ouvrent
- La création du menu de départ et du menu pause
- Et le système de sauvegarde

4.5 Amélioration Possible

Pour les améliorations du projet il y a pour commencer les éléments qui ont été indiqués précédemment dans les erreurs restantes. Ensuite on pourrait varier le comportement des ennemis pour en créer de nouveaux. Par exemple avoir des archers qui nous projettent leurs flèches ou des mages qui lancent leurs boules de feu. Ensuite on pourrait créer une classe PNJ qui pourrait donner plus de vie dans l'univers du jeu en les liant à des quêtes ou en leur faisant raconter une histoire qui pourrait enrichir le lore de l'univers. Et enfin, au lieu de d'animer simplement les déplacements des personnages on pourrait aussi animer les arbres dans le jeu les ennemis les PNJ pour donner vie au monde.

4.6 Liste des documents fournis

Rapport TPI avec Journal de bord format PDF
Readme
Gestion de projet GitProject
Code du jeu sur Github
Code du jeu en fichier ZIP

5 Conclusions

Tout au long de ce projet Pré-TPI j'ai pu avoir un petit aperçu dans la quantité de travail que les développeurs ainsi les designers doivent réaliser pour produire des jeux comme Grand Theft Auto, Hollow Knight ou encore Pokémon, et ce dans un laps de temps avec une technologie beaucoup plus complexe que simplement du Pygame et du PixelArt. J'ai pu observer que sans détermination et passion cela est impossible de continuer à produire un jeu. Cela m'a déjà pris énormément de temps de réaliser un simple de jeu en 2D alors pour faire un jeu en AAA cela doit être extrêmement long même en équipe.
Pour avoir expérimenté moi-même lors de ce Pré-TPI je ne pense pas que la pression infliger aux développeurs pour vite finir un jeu soit justifiable. Cela a été un vrai défi pour ma part dans lequel j'ai pu apprendre pas mal de choses que ce soit dans la réalisation du code, dans la rédaction de document ou même dans la Gestion de Projet dans malgré quelques imprévus ce qui m'a pris pas pris pas mal de temps. Mon projet n'est certes pas entièrement fini mais la majorité des objectifs ont été remplis ce qui représente à mes yeux, une réussite.

6 Annexes

6.1 Glossaire

Lettre	Nom	Définition
A		
	AAA	Le Triple A désigne principalement des à très gros budget, marketing et développement élevés.
B		
	Background	
C		Dans un jeu c'est l'ensemble du contexte narratif, historique et géographique d'un univers ou l'histoire passée d'un personnage, dans notre cela est un contexte géographique.
	Collision	
D		Une collision désigne la détection d'une intersection ou d'un contact physique entre deux objets virtuels.
	Diagramme de Classe	
E		C'est une représentation graphique statique issue de l'UML. Il modélise la structure du jeu en définissant les objets, leurs caractéristiques, leurs comportements et leurs relations.
	Entité	
F		C'est un élément individuel, identifiable de manière unique, qui représente un objet, un personnage, un ennemi ou un composant fonctionnel dans le monde virtuel.
	FPS	
H		FPS (First-Person Shooter) est un genre de jeu vidéo où l'action est vue à travers les yeux du personnage incarnée par le joueur. Le joueur voit ses et ses armes, mais par le reste de son corps, ce qui favorise l'immersion.
	Hitbox	C'est une zone invisible définie autour d'un personnage, d'un objet ou d'un projectile qui détermine les interactions physiques et les dégâts. C'est grâce à cette zone que le jeu calcule si un coup touche sa cible.
	Hp	C'est l'abréviation de l'anglais « Health Points » qui signifie points de vie.
I		
	Inventaire	C'est une interface ou un système (souvent un sac à dos, une mallette ou un menu) permettant au joueur de stocker, organiser, visualiser et utiliser des objets collectés, tels que des armes, consommable ou éléments de quête.
	Item	C'est un objet virtuel collectable, manipulable ou utilisable par un joueur.
M		
	Map	C'est le terrain de jeu, le niveau ou l'espace virtuel dans lequel le joueur évolue et interagit.
	Math.hypot	C'est une fonction qui calcule la racine carrée de la somme des carrés de ses arguments, représentant la norme euclidienne. En développement de jeux, elle est cruciale

		pour calculer rapidement des distances (2D/3D) entre entités, comme la portée d'un tir ou la proximité d'un objet.
	Menu	C'est une interface utilisateur (UI) essentielle servant de panneau de contrôle principal.
	Métroidvania	C'est un sous-genre de jeu d'action-aventure 2D caractérisé par l'exploration d'un vaste monde interconnecté et non linéaire.
P		
	Planification	C'est une fonction exécutive cognitive consistant à organiser mentalement une séquence d'actions stratégiques pour atteindre un but spécifique, comme résoudre un problème ou arriver à destination.
	Pygame	C'est une bibliothèque libre et open-source pour le langage Python, conçue pour créer des jeux vidéo 2D et des applications multimédias.
	PNJ	C'est un personnage dans un jeu vidéo ou un jeu de rôle qui n'est pas contrôlé par le joueur.
R		
	RPG	C'est un RPG est un genre de jeu vidéo où le joueur incarne un ou plusieurs personnages, les faisant évoluer dans un univers narratif.
S		
	Sauvegarde	C'est un fichier informatique enregistrant la progression d'un joueur à un instant précis.
	Sprint	C'est une itération de temps fixe (généralement 2 à 4 semaines) durant laquelle l'équipe réalise un ensemble d'incréments de produit définis à partir du Backlog.
	Sprite	C'est un élément graphique 2D, souvent animé et partiellement transparent, qui se déplace indépendamment sur le fond d'écran dans un jeu vidéo.
T		
	Test unitaire	C'est une procédure de développement logiciel qui vérifie le bon fonctionnement d'une « unité » (fonction, méthode ou classe) de code isolée, indépendamment du reste du programme.
	Test d'intégration	C'est une phase de vérification logicielle qui assemble des modules ou composants individuels pour contrôler leur bon fonctionnement conjoint.
V		
	Vecteur	C'est un objet mathématique représenté par un segment orienté, défini par une direction, un sens et une norme (longueur).

6.2 Résumé du rapport du TPI / version succincte de la documentation

Le monde du jeu vidéo est un univers extrêmement vaste qui fait aujourd'hui, quasiment partie du quotidien plusieurs personnes notamment dans le domaine de l'informatique.

Pour certains c'est un simple loisir tandis que pour d'autres, cela est une passion. Du jeu 2D au 3D, du metroidvania au AAA, du FPS au RPG, chacun arrive à trouver son bonheur dans un jeu et à passer du bon du temps avec.

Une fois arrivé au terme de leur jeu, ils attendent patiemment la sortie d'une suite de ce jeu qui les a faits voyager dans différents univers riches et fantastiques. Pour donner un exemple, le jeu le plus connu et le plus attendu actuellement est Grand Theft Auto VI (GTA 6), qui a fait son annonce il y a maintenant 1 an après 13 ans d'attente ce qui suscite une énorme attente chez les fans.

Pourtant parmi tous ces différents types de jeu, il y a un point qui les relie tous sans exception. Le temps de développement. En effet tout le monde aime bien s'amuser avec son jeu vidéo favori mais personne ne sait réellement combien de temps cela prend de faire ne serait-ce que le déplacement de notre personnage jouable. Quelle est la quantité de travail cela prend à une petite équipe de développement comme Team Cherry par exemple pour développer un jeu Hollow Knight ou comme son successeur Hollow Knight : SilkSong. Combien est-ce que la pression qui est exercé sur les développeurs des géants de l'industrie du jeu vidéo est-elle justifiable ?

C'est en se posant ce genre de question que j'ai décidé de réaliser mon Pré-TPI, et de me lancer moi-même sur la réalisation d'un petit jeu en 2D RPG avec différentes mécaniques de base. Durant le projet je vais non seulement incarner le rôle du développeur mais aussi celui du designeur, pour qu'ainsi je puisse comprendre la charge de travail qui leur est donnée.

6.3 Sources – Bibliographie

https://www.techwithtim.net/tutorials/game-development-with-python/pygame-tutorial/pygame-enemies

https://www.youtube.com/watch?v=whDf00oeI-E&list=PLkkm3wcQHjT7gn81Wn-e78cAyhwBW3FIc&index=7

https://www.youtube.com/watch?v=MRalNWVSqrc

https://www.youtube.com/watch?v=QLPRsWCQ5BE

6.4 Journal de travail

Date	Durée	Activité	Remarques
02.02.2026	4h	Planification du projet	
02.02.2026	30min	Création du joueur	
03.02.2026	1h30	Création du joueur	
03.02.2026	1h30	Début Code Player + plus Premier Background	
03.02.2026	1h30	Documentation	
04.02.2026	1h10	Meeting avec chef de projet	
04.02.2026	20min	Design ennemie	
09.02.2026	20min	Afficher la barre de vie	

09.02.2026	20min	Mouvement du personnage avec ses limites sur l'écran	
09.02.2026	20min	Design Objets	
09.02.2026	1h	Class Enemy	
10.02.2026	2h45	Move enemy	
10.02.2026	3h	Collision	
11.02.2026	2h	Documentation	
23.02.2026	1h30	Documentation plus IceScrum	
24.02.2026	1h30	Documentation + désign des différents objets (épée, potion, clé)	
24.02.2026	2h15	Création de la classe objet plus l'inventaire et commencement du menu principal	
02.03.2026	45 min	Documentation	
02.03.2026	30 min	Changement de gestion de projet : IceScrum -> Git Projects	La raison est que IceScrum c'est bloqué une fois et depuis plus de nouvelle
02.03.2026	1h45	Désign de nouvelles maps	
03.03.2026	1h20	Ajout de la fonction du changement de map	
03.03.2026	1h30	Amélioration des déplacements de l'ennemi (Ajout de son attaque au contacte	
04.03.2026	20 min	Meeting avec le chef de projet	
04.03.2026	20 min	Gestion du planning sur Git Project	
09.03.2026	1h15	Gestion de projet	
09.03.2026	30min	Readme	
10.03.2026	1h30	Création de la classe et mise à jour de l'inventaire	
10.03.2026	45 min	Affichage de l'inventaire plus test Et ramassage des objets ramassables plus test	
10.03.2026	45 min	Affichage de l'image de l'item dans l'inventaire	
11.03.2026	45min	Documentation	
11.03.2026	45min	Meeting avec chef de projet	
11.03.2026	45min	Documentation	
16.03.2026	3h	Documentation	
17.03.2026	45min	Documentation	
17.03.2026	45min	Gestion de l'arme	
17.03.2026	2h15	Reparation de l'inventaire	
18.03.2026	45min	Meeting avec Chef de Projet	
18.03.2026	45 min	Documentation Codes Particuliers	

23.03.2026	1h30	Code Particuliers : Les collisions hitbox	
23.03.2026	45min	Document : Stratégie de Test + Gestion de Projet	
23.03.2026	45min	Changement de langue du code (tout en anglais)	
24.03.2026	1h30	Documentation (Erreurs restantes & Conclusion)	
24.03.2026	2h45	Documentation (Résumé plus amélioration Introduction et Conclusion)	
24.03.2026	5min	Amélioration du changement de map	
24.03.2026	45min	Gestion de Projet (ajout des tests d'interaction du joueur)	
25.03.2026	1h15	Documentation (Légendes & Correction)+	

6.5

6.6 Manuel d'Installation

Prérequis :
Python 3.10 +

Dépendances :
Dans la console installer Pygame

Commande : pip install pygame

6.7 Manuel d'Utilisation

Lancer le fichier main :

Contrôles :
Déplacer à gauche : ←
Déplacer à droite : →
Déplacer en haut : ↑
Déplacer en bas : ↓
L'inventaire : I
Prendre un objet : E
Utiliser un objet dans l'inventaire : E
Equiper une arme dans l'inventaire : E
Déséquiper une arme dans l'inventaire : E
Attaquer : X